Experiment - 7

Motion Estimation using Optical Flow

Lucas-Kanade Sparse Optical Flow

Farneback Dense Optical Flow

In [5]:
```
import cv2
import numpy as np
import imageio.v2 as imageio
from IPython.display import Image, display

cap = cv2.VideoCapture("videoo.mp4")

if not cap.isOpened():
    print("Video not found!")
else:
    ret, old = cap.read()
    old = cv2.resize(old, None, fx=0.4, fy=0.4)
    old_gray = cv2.cvtColor(old, cv2.COLOR_BGR2GRAY)

    p0 = cv2.goodFeaturesToTrack(old_gray, 50, 0.3, 7)
    mask = np.zeros_like(old)
    result = []

    lk = dict(
        winSize=(15, 15),
        maxLevel=1,
        criteria=(cv2.TERM_CRITERIA_EPS |
                  cv2.TERM_CRITERIA_COUNT, 10, 0.03)
    )

    count = 0

    while len(result) < 60:
        ret, frame = cap.read()
        if not ret:
            break

        count += 1

        if count % 3 != 0:
            continue

        frame = cv2.resize(frame, None, fx=0.4, fy=0.4)
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Lucas-Kanade
        if p0 is not None:
            p1, st, _ = cv2.calcOpticalFlowPyrLK(
                old_gray, gray, p0, None, **lk
            )

            if p1 is not None:
                new = p1[st == 1]
                old_pts = p0[st == 1]
```

```
            for n, o in zip(new, old_pts):
                x1, y1 = map(int, n)
                x2, y2 = map(int, o)
                mask = cv2.line(
                    mask, (x1, y1), (x2, y2),
                    (255, 0, 0), 2
                )
                frame = cv2.circle(
                    frame, (x1, y1), 3,
                    (0, 255, 0), -1
                )

            if len(new) > 0:
                p0 = new.reshape(-1, 1, 2)

    # Farneback
    flow = cv2.calcOpticalFlowFarneback(
        old_gray, gray, None,
        0.5, 2, 9, 2, 3, 1.1, 0
    )

    mag, ang = cv2.cartToPolar(
        flow[..., 0], flow[..., 1]
    )

    hsv = np.zeros_like(frame)
    hsv[..., 1] = 255
    hsv[..., 0] = (ang * 180 / np.pi / 2).astype(np.uint8)
    hsv[..., 2] = cv2.normalize(
        mag, None, 0, 255, cv2.NORM_MINMAX
    ).astype(np.uint8)

    dense = cv2.cvtColor(
        hsv, cv2.COLOR_HSV2BGR
    )

    vectors = frame.copy()

    for y in range(0, frame.shape[0], 20):
        for x in range(0, frame.shape[1], 20):
            fx, fy = flow[y, x]
            cv2.arrowedLine(
                vectors, (x, y),
                (int(x + fx), int(y + fy)),
                (0, 255, 255), 1
            )

    lk_result = cv2.add(frame, mask)

    cv2.putText(lk_result, "Lucas-Kanade",
                (10, 25), 0, 0.6, (0,255,255), 2)
    cv2.putText(dense, "Farneback Dense",
                (10, 25), 0, 0.6, (0,255,255), 2)
    cv2.putText(vectors, "Motion Vectors",
                (10, 25), 0, 0.6, (0,255,255), 2)

    combined = np.hstack(
        (lk_result, dense, vectors)
    )
```

```
        result.append(
            cv2.cvtColor(
                combined,
                cv2.COLOR_BGR2RGB
            )
        )

        old_gray = gray.copy()

    cap.release()

    imageio.mimsave(
        "Optical_Flow_Result.gif",
        result,
        duration=0.08,
        loop=0
    )

    print("Experiment completed successfully!")

    display(
        Image(
            filename="Optical_Flow_Result.gif",
            width=1000
        )
    )
```

```
Experiment completed successfully!
<IPython.core.display.Image object>
```

Questions and Answers

1. What is Optical Flow? How is it used in computer vision?

Optical Flow is the apparent motion of objects or pixels between two consecutive video frames. It provides information about the **direction and speed of movement**. In computer vision, it is used for object tracking, motion detection, video stabilization, autonomous driving, and activity recognition.

2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

The Lucas-Kanade algorithm is a **Sparse Optical Flow** method that tracks selected feature points between consecutive frames. It assumes that the motion of pixels within a small window is similar. Using image intensity changes and image gradients, it calculates the displacement of feature points from one frame to the next.

3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Dense Optical Flow calculates motion information for **almost every pixel** in an image. Sparse Optical Flow calculates motion only for selected important feature points. Dense flow provides more complete motion information but generally requires more computational resources than sparse flow.

4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Lucas-Kanade is mainly a **Sparse Optical Flow** method that tracks selected feature points and is computationally efficient. Farneback is a **Dense Optical Flow** method that estimates motion for most pixels in the image. Lucas-Kanade is suitable for feature tracking, while Farneback is useful for detailed motion analysis.

5. What assumptions are made while computing optical flow?

Optical flow generally assumes that the **brightness or intensity of a pixel remains approximately constant** between frames. It also assumes that the motion is relatively small between consecutive frames and that nearby pixels have similar motion. These assumptions make mathematical estimation of motion possible.

6. What factors can affect the accuracy of optical flow estimation?

The accuracy of optical flow can be affected by **lighting changes, fast object motion, camera movement, image noise, occlusion, motion blur, and low-texture regions**. Large changes between frames can also make it difficult to correctly estimate the movement of objects.

7. Mention five real-world applications of optical flow in computer vision.

Five important applications of optical flow are:

1. **Autonomous driving** – Estimating vehicle and road movement.
2. **Video surveillance** – Detecting and tracking moving objects.
3. **Sports analysis** – Tracking players and ball movement.
4. **Gesture recognition** – Analyzing hand and body movements.
5. **Robotics** – Helping robots understand movement in their surroundings.

8. Why are grayscale images generally used for optical flow computation?

Grayscale images are generally used because optical flow mainly depends on **changes in pixel intensity** rather than color information. Converting images to grayscale reduces the amount of data and makes calculations faster while still preserving important information about edges and motion.

9. What are the limitations of optical flow algorithms in real-world environments?

Optical flow can perform poorly when there are **sudden lighting changes, fast motion, occlusions, motion blur, or complex backgrounds**. Camera movement can also produce flow across the entire scene. In addition, some algorithms require significant computational resources for real-time processing.

10. How does optical flow contribute to applications such as autonomous driving, video surveillance, and action recognition?

Optical flow helps **autonomous vehicles** estimate object and camera movement, supports **video surveillance** by detecting and tracking moving objects, and helps **action recognition** by analyzing the motion patterns of people. Therefore, it provides important information for understanding dynamic scenes.

Experiment 7: Optical Flow Tracking

